Syllabus: Introduction to Neural Networks: Biological Neuron, ANN perception, Network Topology (FeedForward networks, Feedback Networks), Adjustments of weights or Learning, Activation Functions, Training Algorithm, Gradient Descent Algorithm- Batch Gradient Descent, Stochastic Gradient

Neural networks are parallel computing devices, which is basically an attempt to make a computer model of the brain. The main objective is to develop a system to perform various computational tasks faster than the traditional systems. These tasks include pattern recognition and classification, approximation, optimization, and data clustering.

What is Artificial Neural Network?

Artificial Neural Network (ANN) is an efficient computing system whose central theme is borrowed from the analogy of biological neural networks. ANNs are also named as "artificial neural systems," or "parallel distributed processing systems," or "connectionist systems." ANN acquires a large collection of units that are interconnected in some pattern to allow communication between the units. These units, also referred to as nodes or neurons, are simple processors which operate in parallel.

Every neuron is connected with other neuron through a connection link. Each connection link is associated with a weight that has information about the input signal. This is the most useful information for neurons to solve a particular problem because the weight usually excites or inhibits the signal that is being communicated. Each neuron has an internal state, which is called an activation signal. Output signals, which are produced after combining the input signals and activation rule, may be sent to other units.

A Brief History of ANN

The history of ANN can be divided into the following three eras −

ANN during 1940s to 1960s

Some key developments of this era are as follows −

- 1943 − It has been assumed that the concept of neural network started with the work of physiologist, Warren McCulloch, and mathematician, Walter Pitts, when in 1943 they modeled a simple neural network using electrical circuits in order to describe how neurons in the brain might work.
- 1949 − Donald Hebb's book, *The Organization of Behavior*, put forth the fact that repeated activation of one neuron by another increases its strength each time they are used.

- 1956 – An associative memory network was introduced by Taylor.
- 1958 – A learning method for McCulloch and Pitts neuron model named Perceptron was invented by Rosenblatt.
- 1960 – Bernard Widrow and Marcian Hoff developed models called "ADALINE" and “MADALINE.”

ANN during 1960s to 1980s

Some key developments of this era are as follows –

- 1961 – Rosenblatt made an unsuccessful attempt but proposed the “backpropagation” scheme for multilayer networks.
- 1964 – Taylor constructed a winner-take-all circuit with inhibitions among output units.
- 1969 – Multilayer perceptron MLPMLP was invented by Minsky and Papert.
- 1971 – Kohonen developed Associative memories.
- 1976 – Stephen Grossberg and Gail Carpenter developed Adaptive resonance theory.

ANN from 1980s till Present

Some key developments of this era are as follows –

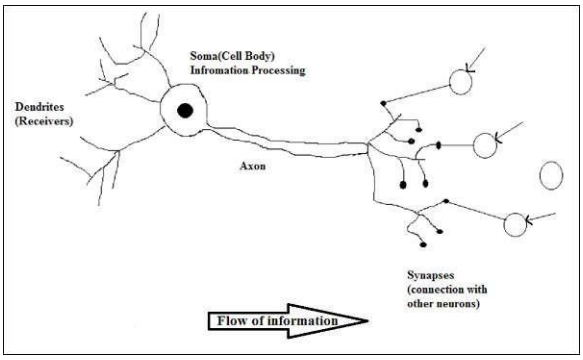
- 1982 – The major development was Hopfield’s Energy approach.
- 1985 – Boltzmann machine was developed by Ackley, Hinton, and Sejnowski.
- 1986 – Rumelhart, Hinton, and Williams introduced Generalised Delta Rule.
- 1988 – Kosko developed Binary Associative Memory BAMBAM and also gave the concept of Fuzzy Logic in ANN.

The historical review shows that significant progress has been made in this field. Neural network based chips are emerging and applications to complex problems are being developed. Surely, today is a period of transition for neural network technology.

Biological Neuron

A nerve cell neuron is a special biological cell that processes information. According to an estimation, there are huge number of neurons, approximately 10^{11} with numerous interconnections, approximately 10^{15}.

Schematic Diagram



Working of a Biological Neuron

As shown in the above diagram, a typical neuron consists of the following four parts with the help of which we can explain its working −

- Dendrites − They are tree-like branches, responsible for receiving the information from other neurons it is connected to. In other sense, we can say that they are like the ears of neuron.
- Soma − It is the cell body of the neuron and is responsible for processing of information, they have received from dendrites.
- Axon − It is just like a cable through which neurons send the information.
- Synapses − It is the connection between the axon and other neuron dendrites.

ANN versus BNN

Before taking a look at the differences between Artificial Neural Network ANN and Biological Neural Network BNN, let us take a look at the similarities based on the terminology between these two.

Biological Neural Network BNN	Artificial Neural Network ANN
Soma	Node
Dendrites	Input
Synapse	Weights or Interconnections
Axon	Output

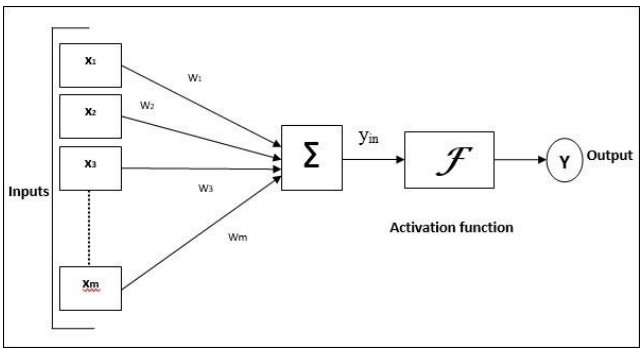
The following table shows the comparison between ANN and BNN based on some criteria mentioned.

Criteria	BNN	ANN
Processing	Massively parallel, slow but superior than ANN	Massively parallel, fast but inferior than BNN
Size	10^{11} neurons and 10^{15} interconnections	10^{2} to 10^{4} nodes mainly depends on the type of application and Network designer
Learning	They can tolerate ambiguity	Very precise, structured and formatted data is required to tolerate ambiguity

Fault tolerance	Performance degrades with even partial damage	It is capable of robust performance, hence has the potential to be fault tolerant
Storage capacity	Stores the information in the synapse	Stores the information in continuous memory locations

Model of Artificial Neural Network

The following diagram represents the general model of ANN followed by its processing.



For the above general model of artificial neural network, the net input can be calculated as follows −

$$y_{in}=x_1.w_1+x_2.w_2+x_3.w_3...x_m.w_m$$

i.e., Net input $y_{in}=\sum_{mi}x_i.w_i$

The output can be calculated by applying the activation function over the net input.

$$Y=F(y_{in})$$

Output = function (net input calculated)

Artificial Neural Network - Building Blocks

Processing of ANN depends upon the following three building blocks −

- Network Topology
- Adjustments of Weights or Learning
- Activation Functions

In this chapter, we will discuss in detail about these three building blocks of ANN

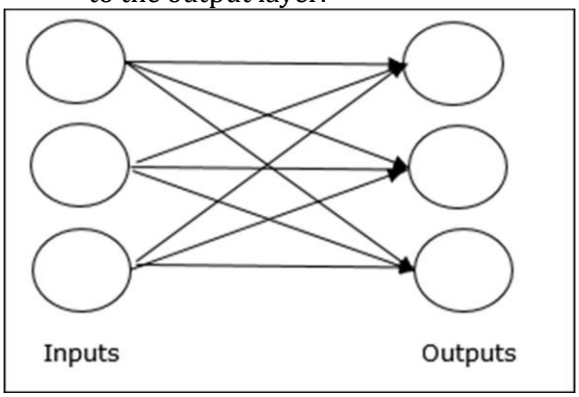
Network Topology

A network topology is the arrangement of a network along with its nodes and connecting lines. According to the topology, ANN can be classified as the following kinds –

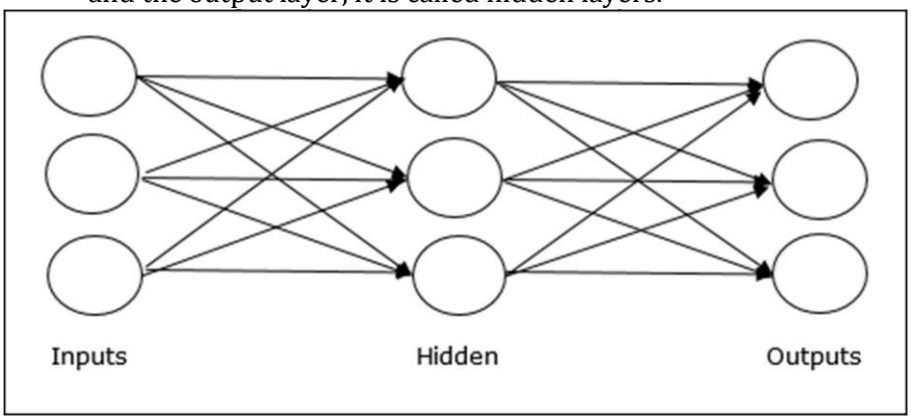
Feedforward Network

It is a non-recurrent network having processing units/nodes in layers and all the nodes in a layer are connected with the nodes of the previous layers. The connection has different weights upon them. There is no feedback loop means the signal can only flow in one direction, from input to output. It may be divided into the following two types –

- **Single layer feedforward network** – The concept is of feedforward ANN having only one weighted layer. In other words, we can say the input layer is fully connected to the output layer.



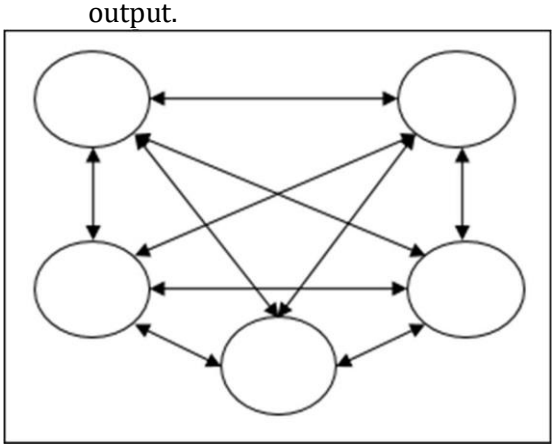
- **Multilayer feedforward network** – The concept is of feedforward ANN having more than one weighted layer. As this network has one or more layers between the input and the output layer, it is called hidden layers.



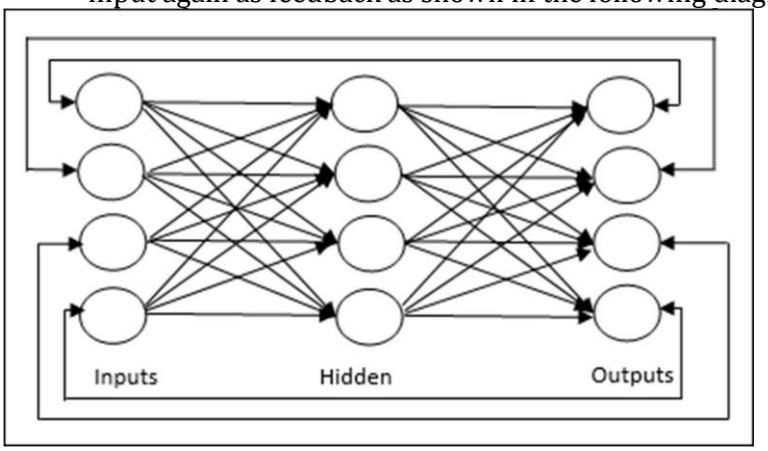
Feedback Network

As the name suggests, a feedback network has feedback paths, which means the signal can flow in both directions using loops. This makes it a non-linear dynamic system, which changes continuously until it reaches a state of equilibrium. It may be divided into the following types –

- **Recurrent networks** – They are feedback networks with closed loops. Following are the two types of recurrent networks.
- **Fully recurrent network** – It is the simplest neural network architecture because all nodes are connected to all other nodes and each node works as both input and output.



- **Jordan network** – It is a closed loop network in which the output will go to the input again as feedback as shown in the following diagram.



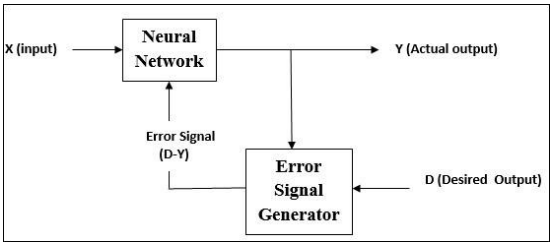
Adjustments of Weights or Learning

Learning, in artificial neural network, is the method of modifying the weights of connections between the neurons of a specified network. Learning in ANN can be classified into three categories namely supervised learning, unsupervised learning, and reinforcement learning.

Supervised Learning

As the name suggests, this type of learning is done under the supervision of a teacher. This learning process is dependent.

During the training of ANN under supervised learning, the input vector is presented to the network, which will give an output vector. This output vector is compared with the desired output vector. An error signal is generated, if there is a difference between the actual output and the desired output vector. On the basis of this error signal, the weights are adjusted until the actual output is matched with the desired output.

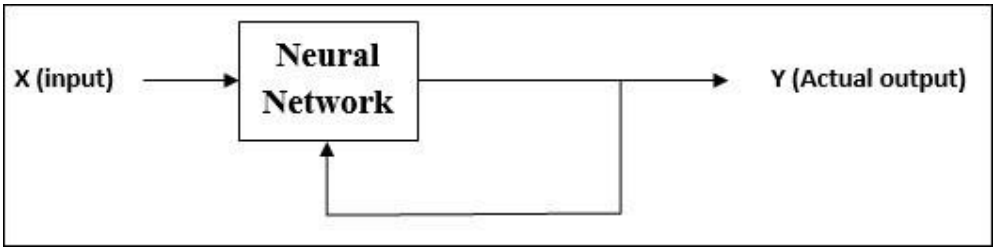


Unsupervised Learning

As the name suggests, this type of learning is done without the supervision of a teacher. This learning process is independent.

During the training of ANN under unsupervised learning, the input vectors of similar type are combined to form clusters. When a new input pattern is applied, then the neural network gives an output response indicating the class to which the input pattern belongs.

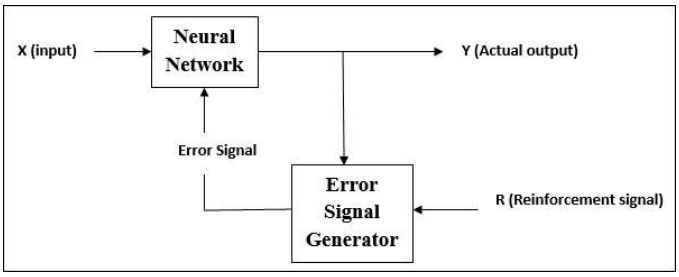
There is no feedback from the environment as to what should be the desired output and if it is correct or incorrect. Hence, in this type of learning, the network itself must discover the patterns and features from the input data, and the relation for the input data over the output.



Reinforcement Learning

As the name suggests, this type of learning is used to reinforce or strengthen the network over some critic information. This learning process is similar to supervised learning, however we might have very less information.

During the training of network under reinforcement learning, the network receives some feedback from the environment. This makes it somewhat similar to supervised learning. However, the feedback obtained here is evaluative not instructive, which means there is no teacher as in supervised learning. After receiving the feedback, the network performs adjustments of the weights to get better critic information in future.

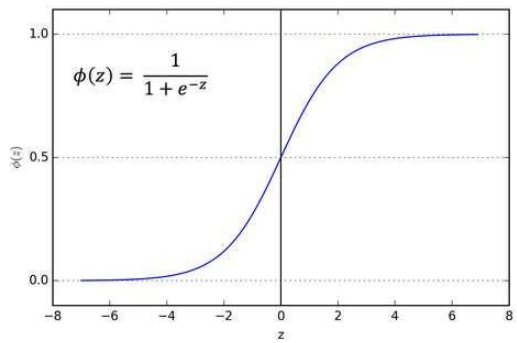


Activation Functions

It may be defined as the extra force or effort applied over the input to obtain an exact output. In ANN, we can also apply activation functions over the input to get the exact output. Followings are some activation functions of interest –

- **Sigmoid Activation Function**

The Sigmoid Function curve looks like a S-shape.



The main reason why we use sigmoid function is because it exists between **(0 to 1).** Therefore, it is especially used for models where we have to **predict the probability** as an output. Since probability of anything exists only between the range of **0 and 1,** sigmoid is the right choice.

The function is **differentiable**. That means, we can find the slope of the sigmoid curve at any two points.

The **Sigmoid** function was introduced to Artificial Neural Networks (ANN) in the 1990s to replace the **Step** function [2]. It was a key change to ANN architecture because the **Step** function doesn't have any gradient to work with Gradient Descent, while the **Sigmoid** function has a well-defined nonzero derivative everywhere, allowing Gradient Descent to make some progress at every step during training.

Problems with Sigmoid activation function

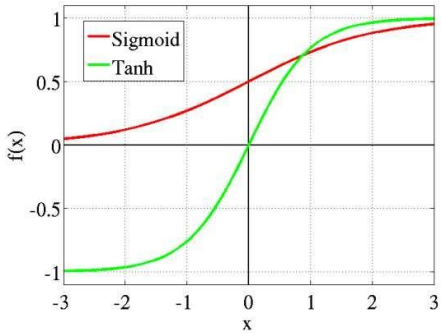
The main problems with the Sigmoid function are:

1. Vanishing gradient: looking at the function plot, you can see that when inputs become small or large, the function saturates at 0 or 1, with a derivative extremely close to 0. Thus it has almost no gradient to propagate back through the network, so there is almost nothing left for lower layers [2].
2. Computationally expensive: the function has an exponential operation.
3. The output is not zero centered:

- **Tanh or hyperbolic tangent Activation Function**

Another very popular and widely used activation function is the Hyperbolic Tangent, also known as Tanh. It is defined as:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



tanh is also like logistic sigmoid but better. The range of the tanh function is from (-1 to 1). tanh is also sigmoidal (s - shaped). The advantage is that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph. The function is **differentiable**. Both tanh and logistic sigmoid activation functions are used in feed-forward nets.

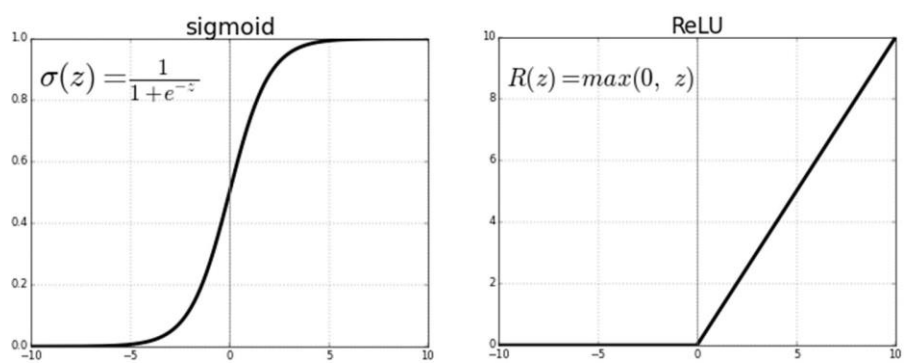
Problems with Tanh activation function

Since Tanh has characteristics similar to Sigmoid, it also faces the following two problems:

1. **Vanishing gradient**: looking at the function plot, you can see that when inputs become small or large, the function saturates at -1 or 1, with a derivative extremely close to 0. Thus it has almost no gradient to propagate back through the network, so there is almost nothing left for lower layers.
2. **Computationally expensive**: the function has an exponential operation.

- **ReLU (Rectified Linear Unit) Activation Function**

The ReLU is the most used activation function in the world right now.Since, it is used in almost all the convolutional neural networks or deep learning.



As you can see, the ReLU is half rectified (from bottom). f(z) is zero when z is less than zero and f(z) is equal to z when z is above or equal to zero.

Range: [0 to infinity)
The function and its derivative both are monotonic.
But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly. That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

Problem with ReLU

ReLU works great in most applications, but it is not perfect. It suffers from a problem known as the dying ReLU.

- Dying ReLU

During training, some neurons effectively die, meaning they stop outputting anything other than 0. In some cases, you may find that half of your network's neurons are dead, especially if you used a large learning rate. A neuron dies when its weights get tweaked in such a way that the weighted sum of its inputs are negative for all instances in the training set. When this happens, it just keeps outputting 0s, and gradient descent does not affect it anymore since the gradient of the ReLU function is 0 when its input is negative.

- Leaky ReLU

It is an attempt to solve the dying ReLU problem

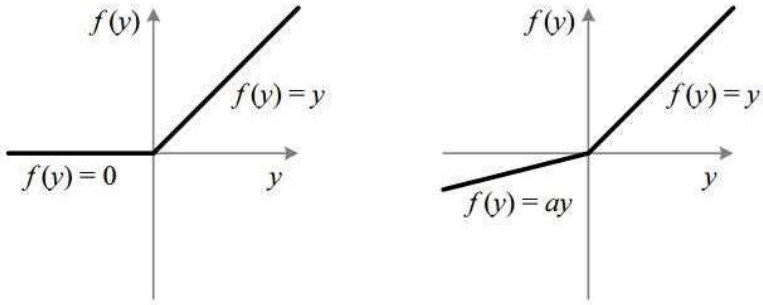


Fig : ReLU v/s Leaky ReLU
Leaky ReLU is defined as:

$f(x) = \max(\alpha x, x)$

Can you see the Leak?

The hyperparameter α defines how much the function leaks. It is the slope of the function for $x < 0$ and is typically set to 0.01. The small slope ensures that Leaky ReLU never dies.The leak helps to increase the range of the ReLU function. Usually, the value of a is 0.01 or so.
When a is not 0.01 then it is called Randomized ReLU.
Therefore the range of the Leaky ReLU is (-infinity to infinity).
Both Leaky and Randomized ReLU functions are monotonic in nature. Also, their derivatives also monotonic in nature.

- ## Exponential Linear Unit (ELU)

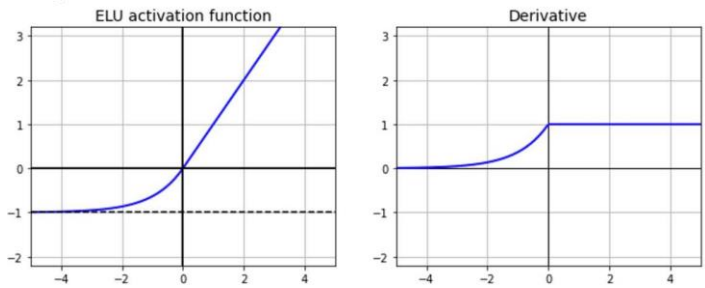
Exponential Linear Unit (ELU) is a variation of ReLU with a better output for z < 0. The function is defined as:

$$\begin{cases} \alpha\left(e^{x}-1\right) & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$$

ELU function (image by author)

The hyperparameter α controls the value to which an ELU saturates for negative net inputs.

The plot of the function and its derivative:



The plot of ELU and its derivative (image by author)

We can see in the plot above,

- ELU modified the slope of the negative part of the function.
- Unlike the Leaky ReLU and PReLU functions, instead of a straight line, ELU uses a log curve for the negative values.

According to the authors, ELU outperformed all the ReLU variants in their experiments

Problem with ELU

The main drawback of the ELU activation is that it is slower to compute than the ReLU and its variants (due to the use of the exponential function), but during training this is compensated by the faster convergence rate. However, at test time, an ELU network will be slower than a ReLU network.

- ## Scaled Exponential Linear Unit (SELU)

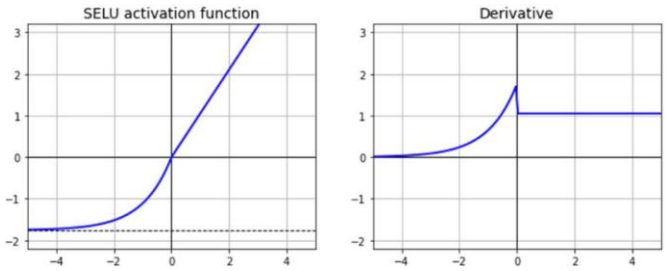
Exponential Linear Unit (SELU) activation function is another variation of ReLU proposed by Günter Klambauer et al. [4] in 2017. The authors showed that if you build a neural network composed exclusively of a stack of dense layers, and if all hidden layers use the SELU activation function, then the network will self-normalize (the output of each layer will tend to preserve mean 0 and standard deviation 1 during training, which resolves the vanishing/exploding gradients problem). This activation function often outperforms other activation functions very significantly.

SELU is defined as:

```
f(x) = scale * x   , z > 0
    = scale * α * (exp(x) - 1)   , z <= 0
```

where α and scale are pre-defined constants (α=1.67326324 and scale=1.05070098).

The plot of SELU and its derivative:



The plot of SELU and its derivative (Image by author)

Problem with SELU

The main problem with SELU is that there are a few conditions for SELU to work:

- SELU works only for a neural network composed exclusively of a stack of dense layers. It might not work for convolutional neural networks.
- Every hidden layer's weights must also be initialized using LeCun normal initialization.
- Input features must be standardized with mean 0 and standard deviation.

Learning and Adaptation

As stated earlier, ANN is completely inspired by the way biological nervous system, i.e. the human brain works. The most impressive characteristic of the human brain is to learn, hence the same feature is acquired by ANN.

What Is Learning in ANN?

Basically, learning means to do and adapt the change in itself as and when there is a change in environment. ANN is a complex system or more precisely we can say that it is a complex adaptive system, which can change its internal structure based on the information passing through it.

Why Is It important?

Being a complex adaptive system, learning in ANN implies that a processing unit is capable of changing its input/output behavior due to the change in environment. The importance of learning in ANN increases because of the fixed activation function as well as the input/output vector, when a particular network is constructed. Now to change the input/output behavior, we need to adjust the weights.

Classification

It may be defined as the process of learning to distinguish the data of samples into different classes by finding common features between the samples of the same classes. For example, to perform training of ANN, we have some training samples with unique features, and to perform its testing we have some testing samples with other unique features. Classification is an example of supervised learning.

Neural Network Learning Rules

We know that, during ANN learning, to change the input/output behavior, we need to adjust the weights. Hence, a method is required with the help of which the weights can

be modified. These methods are called Learning rules, which are simply algorithms or equations. Following are some learning rules for the neural network –

- Hebbian Learning Rule

This rule, one of the oldest and simplest, was introduced by Donald Hebb in his book *The Organization of Behavior* in 1949. It is a kind of feed-forward, unsupervised learning.

Basic Concept – This rule is based on a proposal given by Hebb, who wrote –

"When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."

From the above postulate, we can conclude that the connections between two neurons might be strengthened if the neurons fire at the same time and might weaken if they fire at different times.

Mathematical Formulation – According to Hebbian learning rule, following is the formula to increase the weight of connection at every time step.

$$\Delta w_{ji}(t)=\alpha x_i(t).y_j(t)$$

Here, Δwji(t)= increment by which the weight of connection increases at time step t
α = the positive and constant learning rate
xi(t)= the input value from pre-synaptic neuron at time step t
yi(t)= = the output of pre-synaptic neuron at same time step t

- Perceptron Learning Rule

This rule is an error correcting the supervised learning algorithm of single layer feedforward networks with linear activation function, introduced by Rosenblatt.

Basic Concept – As being supervised in nature, to calculate the error, there would be a comparison between the desired/target output and the actual output. If there is any difference found, then a change must be made to the weights of connection.

Mathematical Formulation – To explain its mathematical formulation, suppose we have 'n' number of finite input vectors, x_n, along with its desired/target output vector t_n, where n = 1 to N.

Now the output 'y' can be calculated, as explained earlier on the basis of the net input, and activation function being applied over that net input can be expressed as follows –

$$y=f(yin)=\begin{cases}1, & yin>\theta \\ 0, & yin\leqslant\theta\end{cases}$$

Where θ is threshold.

The updating of weight can be done in the following two cases –

Case I – when t ≠ y, then

$$w(new)=w(old)+tx$$

Case II – when t = y, then

No change in weight

- Delta Learning Rule Widrow−HoffRule

It is introduced by Bernard Widrow and Marcian Hoff, also called Least Mean Square LMS method, to minimize the error over all training patterns. It is kind of supervised learning algorithm with having continuous activation function.

Basic Concept − The base of this rule is gradient-descent approach, which continues forever. Delta rule updates the synaptic weights so as to minimize the net input to the output unit and the target value.

Mathematical Formulation − To update the synaptic weights, delta rule is given by

$$\Delta w_i = \alpha . x_i . e_j$$

Here Δw_i = weight change for i^{th} pattern;
α = the positive and constant learning rate;
xi = the input value from pre-synaptic neuron;
ej = (t−yin), the difference between the desired/target output and the actual output yin

The above delta rule is for a single output unit only.

The updating of weight can be done in the following two cases −

Case-I − when t ≠ y, then

$$w(new) = w(old) + \Delta w$$

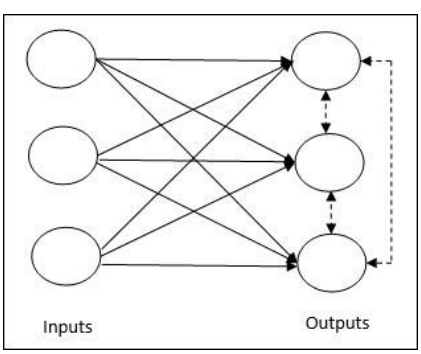
Case-II − when t = y, then

No change in weight

- Competitive Learning Rule Winner−takes− all

It is concerned with unsupervised training in which the output nodes try to compete with each other to represent the input pattern. To understand this learning rule, we must understand the competitive network which is given as follows −

Basic Concept of Competitive Network − This network is just like a single layer feedforward network with feedback connection between outputs. The connections between outputs are inhibitory type, shown by dotted lines, which means the competitors never support themselves.



Basic Concept of Competitive Learning Rule − As said earlier, there will be a competition among the output nodes. Hence, the main concept is that during training, the output unit with the highest activation to a given input pattern, will be declared the winner. This rule

is also called Winner-takes-all because only the winning neuron is updated and the rest of the neurons are left unchanged.

Mathematical formulation − Following are the three important factors for mathematical formulation of this learning rule −

- Condition to be a winner − Suppose if a neuron y_k wants to be the winner then there would be the following condition −

$$y_k = \begin{cases} 1 & \text{if } v_k > v_j \text{ for all } j, j \neq k \\ 0 & \text{otherwise} \end{cases}$$

It means that if any neuron, say y_k, wants to win, then its induced local field the output of summation unit the output of summation unit, say v_k, must be the largest among all the other neurons in the network.

- Condition of sum total of weight − Another constraint over the competitive learning rule is, the sum total of weights to a particular output neuron is going to be 1. For example, if we consider neuron k then −

$$\sum_j w_{kj} = 1 \text{ for all } k$$

Change of weight for winner − If a neuron does not respond to the input pattern, then no learning takes place in that neuron. However, if a particular neuron wins, then the corresponding weights are adjusted as follows

$$\Delta w_{kj} = \begin{cases} -\alpha(x_j - w_{kj}), & \text{if neuron k wins} \\ 0, & \text{if neuron k losses} \end{cases}$$

Here α is the learning rate.

This clearly shows that we are favoring the winning neuron by adjusting its weight and if there is a neuron loss, then we need not bother to re-adjust its weight.

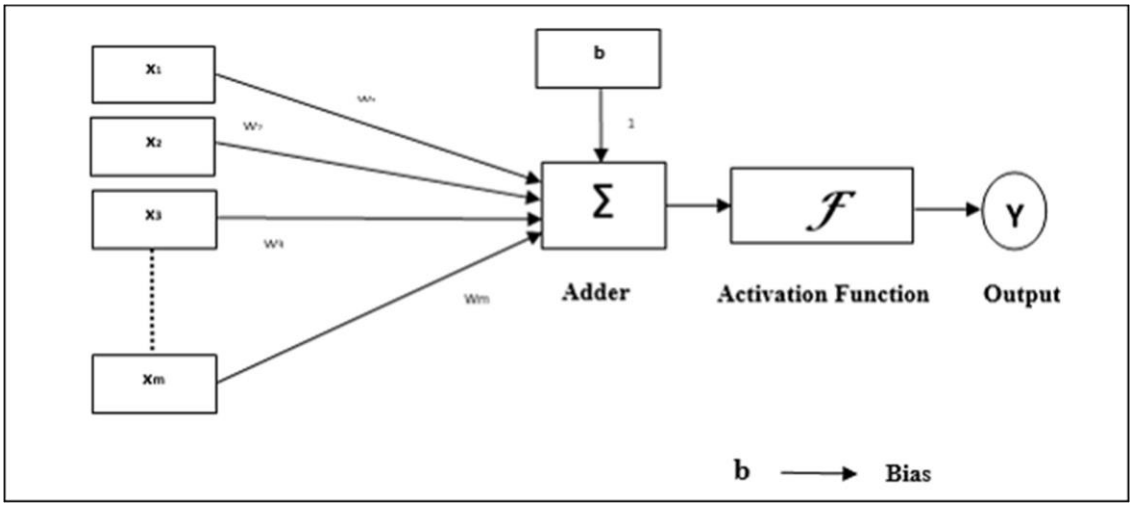
Supervised Learning

As the name suggests, supervised learning takes place under the supervision of a teacher. This learning process is dependent. During the training of ANN under supervised learning, the input vector is presented to the network, which will produce an output vector. This output vector is compared with the desired/target output vector. An error signal is generated if there is a difference between the actual output and the desired/target output vector. On the basis of this error signal, the weights would be adjusted until the actual output is matched with the desired output.

Perceptron

Developed by Frank Rosenblatt by using McCulloch and Pitts model, perceptron is the basic operational unit of artificial neural networks. It employs supervised learning rule and is able to classify the data into two classes.

Operational characteristics of the perceptron: It consists of a single neuron with an arbitrary number of inputs along with adjustable weights, but the output of the neuron is 1 or 0 depending upon the threshold. It also consists of a bias whose weight is always 1. Following figure gives a schematic representation of the perceptron.



Perceptron thus has the following three basic elements −

- **Links** − It would have a set of connection links, which carries a weight including a bias always having weight 1.
- **Adder** − It adds the input after they are multiplied with their respective weights.
- **Activation function** − It limits the output of neuron. The most basic activation function is a Heaviside step function that has two possible outputs. This function returns 1, if the input is positive, and 0 for any negative input.

Training Algorithm

Perceptron network can be trained for single output unit as well as multiple output units.

Training Algorithm for Single Output Unit

Step 1 − Initialize the following to start the training −

- Weights
- Bias
- Learning rate α

For easy calculation and simplicity, weights and bias must be set equal to 0 and the learning rate must be set equal to 1.

Step 2 − Continue step 3-8 when the stopping condition is not true.

Step 3 − Continue step 4-6 for every training vector **x**.

Step 4 − Activate each input unit as follows −

$$x_i=s_i(i=1 \text{to } n)$$

Step 5 − Now obtain the net input with the following relation −

$$y_{in}=b+\sum^{n} \square_i w_i y_{in}$$

Here ‘b’ is bias and ‘n’ is the total number of input neurons.

Step 6 – Apply the following activation function to obtain the final output.

$$(\square_{in}) = \begin{cases} 1 & \square\square\ \square_{in} > \square \\ 0 & \square\square - \square \le \square_{\square\square} \le \square \\ -1 & \square\square\ \square_{in} < \square \end{cases}$$

Step 7 – Adjust the weight and bias as follows –

Case 1 – if y ≠ t then,

$$w_i(new)=w_i(old)+\alpha t_{xi}$$

$$b(new)=b(old)+\alpha t$$

Case 2 – if y = t then,

$$w_i(new)=w_i(old)$$

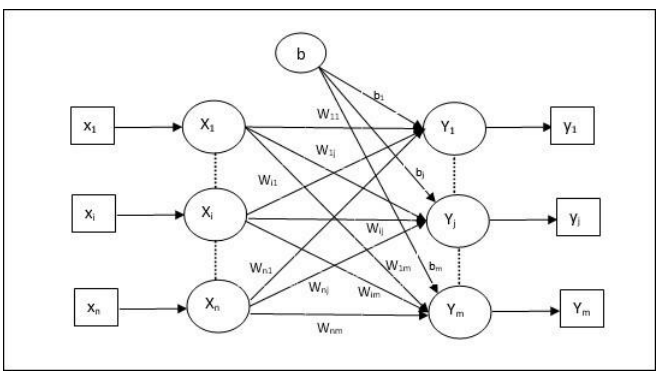
$$b(new)=b(old)$$

Here ‘y’ is the actual output and ‘t’ is the desired/target output.

Step 8 – Test for the stopping condition, which would happen when there is no change in weight.

Training Algorithm for Multiple Output Units

The following diagram is the architecture of perceptron for multiple output classes.



Step 1 – Initialize the following to start the training –

- Weights
- Bias
- Learning rate αα

For easy calculation and simplicity, weights and bias must be set equal to 0 and the learning rate must be set equal to 1.

Step 2 – Continue step 3-8 when the stopping condition is not true.

Step 3 – Continue step 4-6 for every training vector x.

Step 4 – Activate each input unit as follows –

$$x_i=s_i(i=1 \text{ to } n)$$

Step 5 – Obtain the net input with the following relation –

$$y_{in}=b+\sum_i^n$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 6 – Apply the following activation function to obtain the final output for each output unit j = 1 to m –

$$(\square_{in}) = \{0 \square\square - \square \leq \square_{\square\square} \leq \square$$
$$1 \square\square \square_{in} > \square$$
$$-1 \square\square \square_{in} < \square$$

Step 7 – Adjust the weight and bias for x = 1 to n and j = 1 to m as follows –

Case 1 – if $y_j \neq t_j$ then,

$$w_{ij}(new)=w_{ij}(old)+\alpha t_j x_i$$

$$b_j(new)=b_j(old)+\alpha t_j$$

Case 2 – if $y_j = t_j$ then,

$$w_{ij}(new)=w_{ij}(old)w_{ij}(new)=w_{ij}(old)$$

$$b_j(new)=b_j(old)b_j(new)=b_j(old)$$

Here 'y' is the actual output and 't' is the desired/target output.

Step 8 – Test for the stopping condition, which will happen when there is no change in weight.

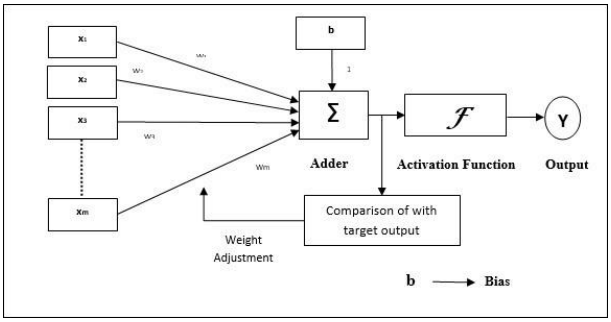
Adaptive Linear Neuron (Adaline)

Adaline which stands for Adaptive Linear Neuron, is a network having a single linear unit. It was developed by Widrow and Hoff in 1960. Some important points about Adaline are as follows –

- It uses bipolar activation function.
- It uses delta rule for training to minimize the Mean-Squared Error (MSE) between the actual output and the desired/target output.
- The weights and the bias are adjustable.

Architecture

The basic structure of Adaline is similar to perceptron having an extra feedback loop with the help of which the actual output is compared with the desired/target output. After comparison on the basis of training algorithm, the weights and bias will be updated.



Training Algorithm

Step 1 – Initialize the following to start the training –

- Weights
- Bias
- Learning rate αα

For easy calculation and simplicity, weights and bias must be set equal to 0 and the learning rate must be set equal to 1.

Step 2 – Continue step 3-8 when the stopping condition is not true.

Step 3 – Continue step 4-6 for every bipolar training pair s:t.

Step 4 – Activate each input unit as follows –

$$x_i=s_i(i=1 \text{to } n)$$

Step 5 – Obtain the net input with the following relation –

$$y_{in}=b+\sum_i^n \square_i \square_i$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 6 – Apply the following activation function to obtain the final output –

$$(\square_{in}) = \begin{cases} 1 \square\square \square_{in} \geq \square \\ -1 \square\square \square_{in} < \square \end{cases}$$

Step 7 – Adjust the weight and bias as follows –

Case 1 – if y ≠ t then,

$$w_i(new)=w_i(old)+\alpha(t-y_{in})x_i w_i(new)=w_i(old)+\alpha(t-y_{in})x_i$$

$$b(new)=b(old)+\alpha(t-yin)b(new)=b(old)+\alpha(t-y_{in})$$

Case 2 – if y = t then,

$$wi(new)=wi(old)wi(new)=wi(old)$$

$$b(new)=b(old)b(new)=b(old)$$

Here 'y' is the actual output and 't' is the desired/target output.

(t−yin) is the computed error.

Step 8 − Test for the stopping condition, which will happen when there is no change in weight or the highest weight change occurred during training is smaller than the specified tolerance.

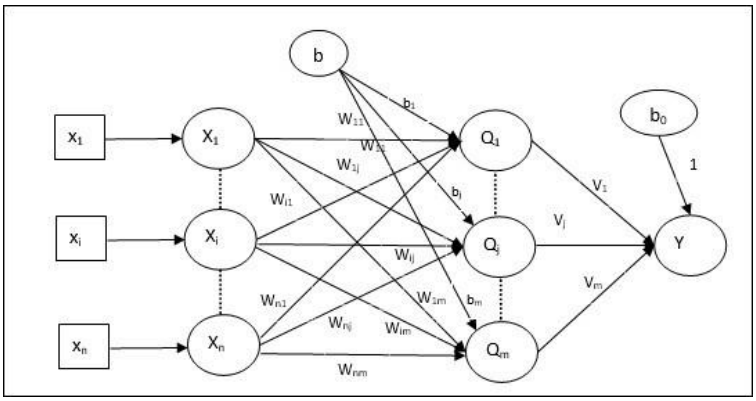
Multiple Adaptive Linear Neuron (Madaline)

Madaline which stands for Multiple Adaptive Linear Neuron, is a network which consists of many Adalines in parallel. It will have a single output unit. Some important points about Madaline are as follows −

- It is just like a multilayer perceptron, where Adaline will act as a hidden unit between the input and the Madaline layer.
- The weights and the bias between the input and Adaline layers, as in we see in the Adaline architecture, are adjustable.
- The Adaline and Madaline layers have fixed weights and bias of 1.
- Training can be done with the help of Delta rule.

Architecture

The architecture of Madaline consists of "n" neurons of the input layer, "m" neurons of the Adaline layer, and 1 neuron of the Madaline layer. The Adaline layer can be considered as the hidden layer as it is between the input layer and the output layer, i.e. the Madaline layer.



Training Algorithm

By now we know that only the weights and bias between the input and the Adaline layer are to be adjusted, and the weights and bias between the Adaline and the Madaline layer are fixed.

Step 1 − Initialize the following to start the training −

- Weights
- Bias
- Learning rate αα

For easy calculation and simplicity, weights and bias must be set equal to 0 and the learning rate must be set equal to 1.

Step 2 − Continue step 3-8 when the stopping condition is not true.

Step 3 – Continue step 4-7 for every bipolar training pair s:t.

Step 4 – Activate each input unit as follows –

$$x_i = s_i (i = 1 \text{ to } n)$$

Step 5 – Obtain the net input at each hidden layer, i.e. the Adaline layer with the following relation –

$$\text{inj} = \text{j} + \sum_{i}^{n} \text{i}\,\text{ij} \quad = 1$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 6 – Apply the following activation function to obtain the final output at the Adaline and the Madaline layer –

$$(\text{in}) = \begin{cases} 1 & \text{in} \geq \\ -1 & \text{in} < \end{cases}$$

Output at the hidden (Adaline) unit

$$Qj = f(Qinj)$$

Final output of the network

$$y = f(y_{in})$$

i.e. $y_{inj} = b_0 + \sum_{j=1}^{m} \text{j}\,\text{j}$

Step 7 – Calculate the error and adjust the weights as follows –

Case 1 – if y ≠ t and t = 1 then,

$$w_{ij}(new) = w_{ij}(old) + \alpha(1 - Q_{inj})$$

$$b_j(new) = b_j(old) + \alpha(1 - Q_{inj})$$

In this case, the weights would be updated on Q_j where the net input is close to 0 because t = 1.

Case 2 – if y ≠ t and t = -1 then,

$$w_{ik}(new) = w_{ik}(old) + \alpha(-1 - Q_{ink})x_i$$

$$b_k(new) = b_k(old) + \alpha(-1 - Q_{ink})$$

In this case, the weights would be updated on Q_k where the net input is positive because t = -1.

Here 'y' is the actual output and 't' is the desired/target output.

Case 3 – if y = t then

There would be no change in weights.

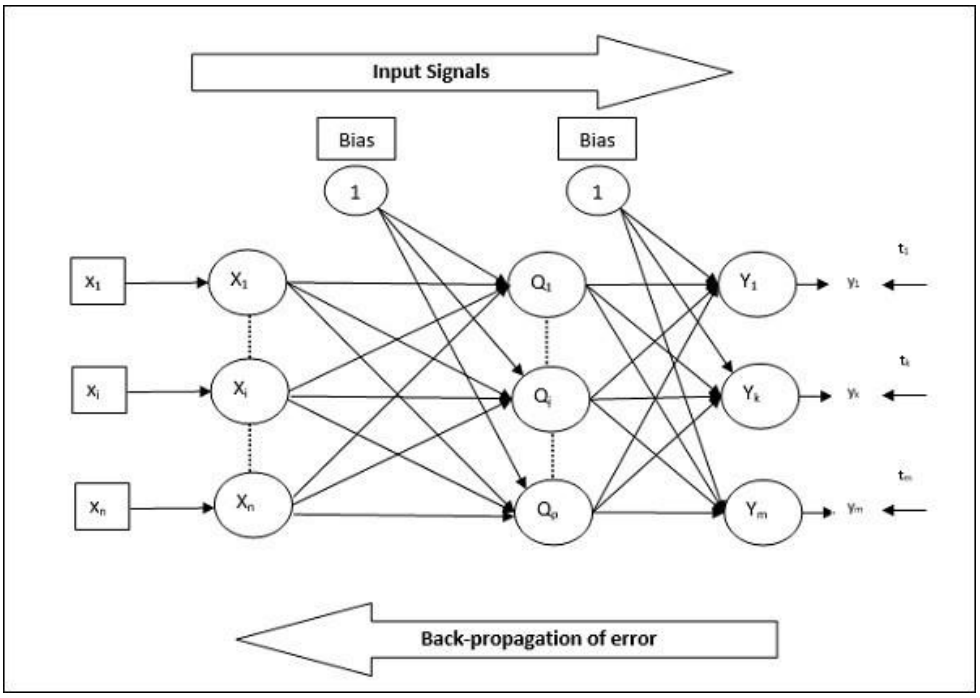
Step 8 – Test for the stopping condition, which will happen when there is no change in weight or the highest weight change occurred during training is smaller than the specified tolerance.

Back Propagation Neural Networks

Back Propagation Neural (BPN) is a multilayer neural network consisting of the input layer, at least one hidden layer and output layer. As its name suggests, back propagating will take place in this network. The error which is calculated at the output layer, by comparing the target output and the actual output, will be propagated back towards the input layer.

Architecture

As shown in the diagram, the architecture of BPN has three interconnected layers having weights on them. The hidden layer as well as the output layer also has bias, whose weight is always 1, on them. As is clear from the diagram, the working of BPN is in two phases. One phase sends the signal from the input layer to the output layer, and the other phase back propagates the error from the output layer to the input layer.



Training Algorithm

For training, BPN will use binary sigmoid activation function. The training of BPN will have the following three phases.

- Phase 1 – Feed Forward Phase
- Phase 2 – Back Propagation of error
- Phase 3 – Updating of weights

All these steps will be concluded in the algorithm as follows

Step 1 – Initialize the following to start the training –

- Weights
- Learning rate αα

For easy calculation and simplicity, take some small random values.

Step 2 – Continue step 3-11 when the stopping condition is not true.

Step 3 – Continue step 4-10 for every training pair.

Phase 1

Step 4 – Each input unit receives input signal x_i and sends it to the hidden unit for all i = 1 to n

Step 5 – Calculate the net input at the hidden unit using the following relation –

$$Q_{inj} = b_{0j} + \sum_{i=1}^{n} x_i v_{ij} \quad (j = 1 \text{ to } p)$$

Here b_{0j} is the bias on hidden unit, v_{ij} is the weight on j unit of the hidden layer coming from i unit of the input layer.

Now calculate the net output by applying the following activation function

$$Q_j = f(Q_{inj})$$

Send these output signals of the hidden layer units to the output layer units.

Step 6 – Calculate the net input at the output layer unit using the following relation –

$$y_{ink} = b_{0j} + \sum_{j=1}^{p} Q_j w_{jk} \quad (k = 1 \text{ to } m)$$

Here b_{0k} is the bias on output unit, w_{jk} is the weight on k unit of the output layer coming from j unit of the hidden layer.

Calculate the net output by applying the following activation function

$$y_k = f(y_{ink})$$

Phase 2

Step 7 – Compute the error correcting term, in correspondence with the target pattern received at each output unit, as follows –

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

On this basis, update the weight and bias as follows –

$$\Delta v_{jk} = \alpha \delta_k Q_{ij}$$

$$\Delta b_{0k} = \alpha \delta_k$$

Then, send δ_k back to the hidden layer.

Step 8 – Now each hidden unit will be the sum of its delta inputs from the output units.

$$\delta_{inj}=\sum_{k=1}^{m} \delta_k \square_{jk}$$

Error term can be calculated as follows –

$$\delta_j=\delta_{inj} f'(Q_{inj})$$

On this basis, update the weight and bias as follows –

$$\Delta w_{ij}=\alpha \delta_j x_i$$

$$\Delta b_{0j}=\alpha \delta_j$$

Phase 3

Step 9 – Each output unit *($y_k k$ = 1 to m)* updates the weight and bias as follows –

$$v_{jk}(new)=v_{jk}(old)+\Delta v_{jk}$$

$$b_{0k}(new)=b_{0k}(old)+\Delta b_{0k}$$

Step 10 – Each output unit (zjj = 1 to p) updates the weight and bias as follows –

$$w_{ij}(new)=w_{ij}(old)+\Delta w_{ij}$$

$$b_{0j}(new)=b_{0j}(old)+\Delta b_{0j}$$

Step 11 – Check for the stopping condition, which may be either the number of epochs reached or the target output matches the actual output.

Generalized Delta Learning Rule

Delta rule works only for the output layer. On the other hand, generalized delta rule, also called as back-propagation rule, is a way of creating the desired values of the hidden layer.

Mathematical Formulation

For the activation function $y_k=f(y_{ink})$ the derivation of net input on Hidden layer as well as on output layer can be given by

$$\square_{ink} = \sum_{i} \square_i \square_{jk}$$

And

$$\square_{inj} = \sum_{i} \square_i \square_{ij}$$

Now the error which has to be minimized is

$$\square = \frac{1}{2} \sum_{k} [\square_k - \quad]_k^2$$

By using the chain rule, we have

$$\frac{\delta E}{\delta w_{jk}} = \frac{\delta}{\delta w_{jk}} \left(\frac{1}{2}\sum_{k} [t_k - y_k]^2\right)$$

$$= \frac{\delta}{\delta w_{jk}} \left(\frac{1}{2}\sum_{k} [t_k - y_k]^2\right)$$

$$= [t_k - y_k] \frac{\delta}{\delta w_{jk}} (f(y_{ink}))$$

$$= [t_k - y_k] f'(y_{ink}) \frac{\delta}{\delta w_{jk}} (y_{ink})$$

$$= [t_k - y_k]' (y_{ink}) z_j$$

Now let us say $\delta_k = -[t_k - y_k] f'(y_{ink})$

The weights on connections to the hidden unit z_j can be given by –

$$\frac{\delta E}{\delta v_i} = -\sum_k \delta_k \frac{\delta}{\delta v_{ij}} (y_{ink})$$

Putting the value of y_{ink} we will get the following

$$\delta_j = -\sum_k \delta_k w_{jk} f'(y_{inj})$$

Weight updating can be done as follows –

For the output unit –

$$\Delta w_{jk} = -\alpha \frac{\delta E}{\delta w_{jk}}$$

$$= \alpha \delta_k$$

For the hidden unit –

$$\Delta v_{ij} = -\alpha \frac{\delta E}{\delta v_{ij}}$$

$$= \alpha \delta_j x_i$$

Gradient Descent Algorithm

The Gradient Descent is an optimization algorithm which is used to minimize the cost function for many machine learning algorithms. Gradient Descent algorithm is used for updating the parameters of the learning models.

Following are the different types of Gradient Descent:

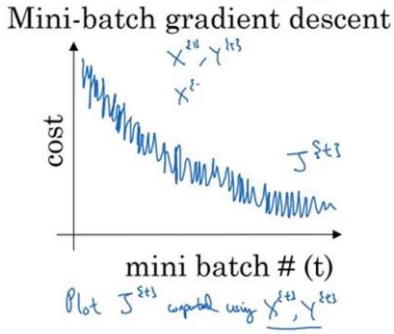
Batch Gradient Descent: The Batch Gradient Descent is the type of Gradient Algorithm that is used for processing all the training datasets for each iteration of the gradient descent. Suppose the number of the training dataset is large, the batch gradient descent will be comparatively expensive. Hence, if the number of the training dataset is large, the users are not advised to use batch gradient descent. Instead, they can use mini-batch gradient descent for a large training dataset.

Mini-Batch Gradient Descent: We have seen the Batch Gradient Descent. We have also seen the Stochastic Gradient Descent. Batch Gradient Descent can be used for smoother curves. SGD can be used when the dataset is large. Batch Gradient Descent converges directly to minima. SGD converges faster for larger datasets. But, since in SGD we use only one example at a time, we cannot implement the vectorized implementation on it. This can slow down the computations. To tackle this problem, a mixture of Batch Gradient Descent and SGD is used.

Neither we use all the dataset all at once nor we use the single example at a time. We use a batch of a fixed number of training examples which is less than the actual dataset and call it a mini-batch. Doing this helps us achieve the advantages of both the former variants we saw. So, after creating the mini-batches of fixed size, we do the following steps in **one epoch:**

1. Pick a mini-batch
2. Feed it to Neural Network
3. Calculate the mean gradient of the mini-batch
4. Use the mean gradient we calculated in step 3 to update the weights
5. Repeat steps 1–4 for the mini-batches we created

Just like SGD, the average cost over the epochs in mini-batch gradient descent fluctuates because we are averaging a small number of examples at a time.



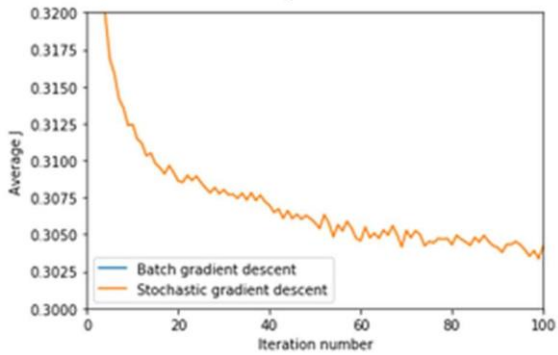
Cost vs no of mini-batch

So, when we are using the mini-batch gradient descent we are updating our parameters frequently as well as we can use vectorized implementation for faster computations.

Stochastic Gradient Descent: Stochastic gradient descent is the type of gradient descent which can process one training dataset per iteration. Therefore, the parameters will be updated after each iteration, in which only one dataset has been processed. This type of gradient descent is faster than the Batch Gradient Descent. But, if the number of training

datasets is large then also, it will process only one dataset at a time. Therefore, the number of iterations will be large.
We do the following steps in one epoch for SGD:

1. Take an example
2. Feed it to Neural Network
3. Calculate it's gradient
4. Use the gradient we calculated in step 3 to update the weights
5. Repeat steps 1–4 for all the examples in training dataset
6. Since we are considering just one example at a time the cost will fluctuate over the training examples and it will not necessarily decrease. But in the long run, you will see the cost decreasing with fluctuations.



Variables used:

Let **'k'** be the number of training datasets.

Let **'j'** be the number of features in the dataset.

If **p == k**, the mini-batch gradient descent will behave similarly to the batch gradient descent. (where **'p'** is batch gradient descent)

Algorithm used for batch gradient descent:

Let $\mathbf{h_\theta(a)}$ be the hypothesis for linear regression. Then the cost function will be given by:

Let **Σ** represents the sum of all the training datasets from **t = 1** to **k**.

$$G_{train}(\theta) = (1/2k)\ \Sigma\ (h_\theta(a^{(t)}) - b^{(t)})^2$$

Repeat {

$\theta g = \theta g - (\text{learning rate}/k) * \Sigma\ (h_\theta(a^{(t)}) - b^{(t)})a_g^{(t)}$

For every g = 0 …j

}

Where $a_g^{(t)}$ represents the gth feature of the tth training dataset, suppose if 'k' is very large (for example, 7 million number of training datasets), then batch gradient descent will take hours or maybe days for completing the process. Therefore, for large training datasets, batch gradient descent is not recommended to the users as this will slows down the learning process of the machine.

Algorithm used for mini-batch gradient descent

Suppose 'p' is the number of datasets in one batch, where $p < k$.
Let $p = 10$ and $k = 100$;
However the users can adjust the batch size. This is generally written as a power of 2.

Repeat {
For t = 1, 11, 21, ….., 91

Let Σ be the summation from t to t+9 represented by d.
$\Theta_g = \theta_g$ - (learning rate/size of (p)) * $\Sigma\ (h_\theta(a^{(d)}) - b^{(d)})a_g^{(d)}$
For every g = 0 …j
}

Algorithm used for stochastic gradient descent:

- This gradient descent will randomly shuffle the dataset for training the parameters for each kind of data.
- The stochastic gradient descent will take only one dataset per iteration.

Hence,
Let $(a^{(t)}, b^{(t)})$ be the training dataset
$\text{Cost}(\theta, (a^{(t)}, b^{(t)})) = (1/2)\ \Sigma\ (h\theta(a^{(t)}) - b^{(t)})^2$

$G_{train}(\theta) = (1/k)\ \Sigma\ \text{Cost}\ (\theta, (a^{(t)}, b^{(t)}))$

Repeat {
For t = 1 to k{
$\Theta_g = \theta_g$ - (learning rate) * $\Sigma\ (h_\theta(a^{(t)}) - b^{(t)})a_g^{(t)}$
For every g = 0 …j

}
}

Numerical Stability and Initialization

- **N u m e r i c a l S t N u**

Thus far, every model that we have implemented required that we initialize its parameters according to some pre-specified distribution. Until now, we took the initialization scheme for granted, glossing over the details of how these choices are made. You might have even gotten the impression that these choices are not especially important. On the contrary, the choice of initialization scheme plays a significant role in neural network learning, and it can be crucial for maintaining numerical stability. Moreover, these choices can be tied up in interesting ways with the choice of the nonlinear activation function. Which function we choose and how we initialize parameters can determine how quickly our optimization algorithm converges. Poor choices here can cause us to encounter exploding or vanishing gradients while training. In this section, we delve into these topics in greater detail and discuss some useful heuristics that you will find useful throughout your career in deep learning.

```
%matplotlib inline
import torch
from d21 import torch as d21
```

5..1 Vanishing and Exploding Gradients

Consider a deep network with L layers, input x and output o. With each layer l defined by a transformation Ji parametrized by weights $wU)$, whose hidden layer output is **h** Cl) (let h(O) = x), our network can be expressed as:

$$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)}) \text{ and thus } \mathbf{o} = f_L \circ \cdots \circ f_1(\mathbf{x}). \tag{5.4.1}$$

Tf all the hidden layer output and the input are vectors, we can write the gradient of o with respect to any set of parameters w<0 as follows:

$$awuio = \text{ah(L-J)h(L)} \cdot .. \text{ahulh(/+I) aw(I)h(/)}. \tag{5.4.2}$$

In other words, this gradient is the product of L - l matrices M(L) • • • M(I+I) and the gradient vector vU). Thus we are susceptible to the same problems of numerical underflow that often crop up when multiplying together too many probabilities. When dealing with probabilities, a common trick is to switch into log-space, i.e., shifting pressure from the

mantissa to the exponent of the numerical representation. Unfortunately, our problem above is more serious: initially the matrices $M^{(t)}$ may have a wide variety of eigenvalues. They might be small or large, and their product might be *very large* or *very small*.

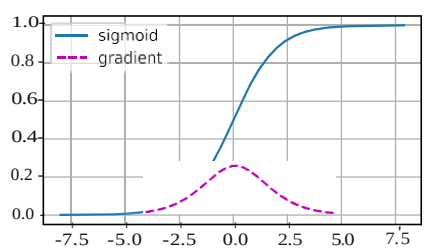
The risks posed by unstable gradients go beyond numerical representation. Gradients of unpredictable magnitude also threaten the stability of our optimization algorithms. We may be facing parameter updates that are either (i) excessively large, destroying our model (the *exploding gradient* problem); or (ii) excessively small (the *vanishing gradient* problem), rendering learning impossible as parameters hardly move on each update.

Vanishing Gradients

One frequent culprit causing the vanishing gradient problem is the choice of the activation function σ that is appended following each layer's linear operations. Historically, the sigmoid function $1/(1+\exp(-x))$ (introduced in Section 5.1) was popular because it resembles a thresholding function. Since early artificial neural networks were inspired by biological neural networks, the idea of neurons that fire either fully or *not at all* (like biological neurons) seemed appealing. Let's take a closer look at the sigmoid to see why it can cause vanishing gradients.

```
x = torch.arange(-8.0, 8.0, 0.1, requires_grad=True)
y = torch.sigmoid(x)
y.backward(torch.ones_like(x))

d2l.plot(x.detach().numpy(),[y.detach().numpy(), x.grad.numpy()],
         legend=['sigmoid', 'gradient'], figsize=(4.5, 2.5))
```



As you can see, the sigmoid's gradient vanishes both when its inputs are large and when they are small. Moreover, when backpropagating through many layers, unless we are in the Goldilocks zone, where the inputs to many of the sigmoids are close to zero, the gradients of the overall product may vanish. When our network boasts many layers, unless we are careful, the gradient will likely be cut off at some layer. Indeed, this problem used to plague deep network training. Consequently, ReLUs, which are more stable (but less neurally plausible), have emerged as the default choice for practitioners.

Exploding Gradients

The opposite problem, when gradients explode, can be similarly vexing. To illustrate this a bit better, we draw 100 Gaussian random matrices and multiply them with some initial matrix. For the scale that we picked (the choice of the variance $r.r^2 = 1$), the matrix product explodes. When this happens because of the initialization of a deep network, we have no chance of getting a gradient descent optimizer to converge.

```
M = torch.normal(0, 1, size=(4, 4))
print('a single matrix \n',M)
for i in range(100):
    M = M@torch.normal(0 , 1, size= (4,4))
print('after multiplying 100 matrices\n', M)
```

```
a single matrix
 tensor([[-0.8755, -1.2171, 1.3316, 0.1357],
        [ 0.4399, 1.4073, -1.9131, -0.4608],
        [-2.1420, 0.3643,-0.5267, 1.0277],
        [-0.1734, -0.7549, 2.3024, 1.3085]])
after multiplying 100 matrices
 tensor([[-2.9185e+23, 1.3915e+25, -1.1865e+25, 1.4354e+24],
        [ 4.9142e+23, -2.3430e+25, 1.9979e+25, -2.4169e+24],
        [ 2.6578e+23, -1.2672e+25, 1.0805e+25, -1.3072e+24],
        [-5.2223e+23, 2.4899e+25, -2.1231e+25, 2.5684e+24]])
```

Breaking the Symmetry

Another problem in neural network design is the symmetry inherent in their parametrization. Assume that we have a simple MLP with one hidden layer and two units. In this case, we could permute the weights WCI) of the first layer and likewise permute the weights of the output layer to obtain the same function. There is nothing special differentiating the first and second hidden units. ln other words, we have permutation symmetry among the hidden units of each layer.

This is more than just a theoretical nuisance. Consider the aforementioned one-hidden-layer MLP with two hidden units. For illustration, suppose that the output layer transforms the two hidden units into only one output unit. Imagine what would happen if we initialized all the parameters of the hidden layer as wO) = c for some constant c. In this case, during forward propagation either hidden unit takes the same inputs and parameters producing the same activation which is fed to the output unit. During backpropagation, differentiating the output unit with respect to parameters **W<**[1]) gives a gradient all of whose elements take the same value. Thus, after gradient-based iteration (e.g., minibatch stochastic gradient descent), all the elements ofw(t) still take the same value. Such iterations would never *break the symmelry* on their own and we might never be able to realize the network's expressive power. The hidden layer would behave as if it had only a single unit. Note that while minibatch stochastic gradient descent would not break this symmetry, dropout regularization (to be introduced later) would!

5..2 Parameter Initialization

One way of addressing-or at least mitigating-the issues raised above is through careful initialization. As we will see later, additional care during optimization and suitable regularization can further enhance stability.

Default Initialization

In the previous sections, e.g., in Section 3.5, we used a normal distribution to initialize the values of our weights. If we do not specify the initialization method, the framework will use a default random initialization method, which often works well in practice for moderate problem sizes.

Xavier Initialization

Let's look at the scale distribution of an output a; for some fully connected layer *without nunlinearitie:*,. With nin inputs *x_;* and their associated weights *w;_;* for this layer, an output is given by

$$Oi = \sum_{i=I}^{flin} LW;_;X_;. \tag{5.4.3}$$

The weights *w;_;* are all drawn independently from the same distribution. Furthermore, let's assume that this distribution has zero mean and variance cr^2. Note that this does not mean that the distribution has to be Gaussian, just that the mean and variance need to exist. For now, let's assume that the inputs to the layer *x_;* also have zero mean and variance y^2 and that they are independent of *wu* and independent of each other. In this case, we can compute the mean of a;:

$$\begin{aligned} E[o;] &= \sum_{i=I}^{1i11} E[w;;x_;] \\ &= \sum_{i=I}^{flin} E[w;_;]E[x_;] \\ &= 0, \end{aligned} \tag{5.4.4}$$

and the variance:

$$\begin{aligned} \mathrm{Var}[a;] &= E[a\}] - (E[a;])^2 \\ &= \sum_{i=I}^{Ilin} E[wtxJ] - 0 \\ &= \sum_{i=I}^{IL.in} E[w7_T]E[x^2]_I \\ &= 11inCT^2Y^2. \end{aligned} \tag{5.4.5}$$

One way to keep the variance fixed is to set $11iacr^2 = 1$. Now consider backpropagation. There we face a similar problem, albeit with gradients being propagated from the layers

closer to the output. Using the same reasoning as for forward propagation, we see that the gradients' variance can blow up unless $n_0 u_1 \varepsilon_T^2 = 1$, where *nout* is the number of outputs of this layer. This leaves us in a dilemma: we cannot possibly satisfy both conditions simultaneously. Instead, we simply try to satisfy:

$$\frac{1}{2}(n_{in} + n_0 u_1)\varepsilon_T^2 = 1 \text{ or equivalently } \varepsilon_T = \sqrt{\frac{2}{n_{in} + n_{out}}}. \tag{5.4.6}$$

This is the reasoning underlying the now-standard and practically beneficial *Xavier initialization,* named after the first author of its creators (Glorot and Bengio, 2010). Typically, the Xavier initialization samples weights from a Gaussian distribution with zero mean and variance $\varepsilon_T^2 = \frac{2}{n_{in} + n_{out}}$. We can also adapt this to choose the variance when sampling weights from a uniform distribution. Note that the uniform distribution *U(-a,a)* has variance $\frac{(a-(-a))^2}{12} = \frac{a^2}{3}$. Plugging $\frac{a^2}{3}$ into our condition on ε_T^2 prompts us to initialize according to

$$U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right). \tag{5.4.7}$$

Though the assumption for nonexistence of nonlinearities in the above mathematical reasoning can be easily violated in neural networks, the Xavier initialization method turns out to work well in practice.

Beyond

The reasoning above barely scratches the surface of modern approaches to parameter initialization. A deep learning framework often implements over a dozen different heuristics. Moreover, parameter initialization continues to be a hot area of fundamental research in deep learning. Among these are heuristics specialized for tied (shared) parameters, super-resolution, sequence models, and other situations. For instance, Xiao *et al.* (2018) demonstrated the possibility of training 10,000-layer neural networks without architectural tricks by using a carefully-designed initialization method.

If the topic interests you we suggest a deep dive into this module's offerings, reading the papers that proposed and analyzed each heuristic, and then exploring the latest publications on the topic. Perhaps you will stumble across or even invent a clever idea and contribute an implementation to deep learning frameworks.

5..3 Summary

Vanishing and exploding gradients are common issues in deep networks. Great care in parameter initialization is required to ensure that gradients and parameters remain well controlled. initialization heuristics are needed to ensure that the initial gradients are neither too large nor too small. Random initialization is key to ensuring that symmetry is broken before optimization. Xavier initialization suggests that, for each layer, variance of any output is not affected by the number of inputs, and variance of any gradient is not affected by the number of outputs. ReLU activation functions mitigate the vanishing gradient problem. This can accelerate convergence.

5..4 Exercises

1. Can you design other cases where a neural network might exhibit symmetry that needs breaking, besides the permutation symmetry in an MLP's layers?
2. Can we initialize all weight parameters in linear regression or in softmax regression to the same value?
3. Look up analytic bounds on the eigenvalues of the product of two matrices. What does this tell you about ensuring that gradients are well conditioned?
4. If we know that some terms diverge, can we fix this after the fact? Look at the paper on layerwise adaptive rate scaling for inspiration (You *el al.,* 2017).

105 Discussions [105].

• Generalization in Deep Learning

In Chapter 3 and Chapter 4, we tackled regression and classification problems by fitting linear models to training data. In both cases, we provided practical algorithms for finding the parameters that maximized the likelihood of the observed training labels. And then, towards the end of each chapter, we recalled that fitting the training data was only an intermediate goal. Our real quest all along was to discover *general patterns* on the basis of which we can make accurate predictions even on new examples drawn from the same underlying population. Machine learning researchers are *con!>umers* of optimization algorithms. Sometimes, we must even develop new optimization algorithms. But at the end of the day, optimization is merely a means to an end. At its core, machine learning is a statistical discipline and we wish to optimize training loss only insofar as some statistical principle (known or unknown) leads the resulting models to generalize beyond the training set.

On the bright side, it turns out that deep neural networks trained by stochastic gradient descent generalize remarkably well across myriad prediction problems, spanning computer vision; natural language processing; time series data; recommender systems; electronic health records; protein folding; value function approximation in video games and board games; and numerous other domains. On the downside, if you were looking for a straightforward account of either the optimization story (why we can fit them to training data) or the generalization story (why the resulting models generalize to unseen examples), then you might want to pour yourself a drink. While our procedures for optimizing linear models and the statistical properties of the solutions are both described well by a comprehensive body of theory, our understanding of deep learning still resembles the wild west on both fronts.

Both the theory and practice of deep learning are rapidly evolving, with theorists adopting new strategies to explain what's going on, even as practitioners continue to innovate at

a blistering pace, building arsenals of heuristics for training deep networks and a body of intuitions and folk knowledge that provide guidance for deciding which techniques to apply in which situations.

The summary of the present moment is that the theory of deep learning has produced promising lines of attack and scattered fascinating results, but still appears far from a comprehensive account of both (i) why we are able to optimize neural networks and (ii) how models learned by gradient descent manage to generalize so well, even on high-dimensional tasks. However, in practice, (i) is seldom a problem (we can always find parameters that will fit all of our training data) and thus understanding generalization is far the bigger problem. On the other hand, even absent the comfort of a coherent scientific theory, practitioners have developed a large collection of techniques that may help you to produce models that generalize well in practice. While no pithy summary can possibly do justice to the vast topic of generalization in deep learning, and while the overall state of research is far from resolved, we hope, in this section, to present a broad overview of the state of research and practice.

5..1 Revisiting Overfitting and Regularization

According to the "no free lunch" theorem of Wolpert and Macready (1995), any learning algorithm generalizes better on data with certain distributions, and worse with other distributions. Thus, given a finite training set, a model relies on certain assumptions: to achieve human-level performance it may be useful to identify *inductive hiases* that reflect how humans think about the world. Such inductive biases show preferences for solutions with certain properties. For example, a deep MLP has an inductive bias towards building up a complicated function by the composition of simpler functions.

With machine learning models encoding inductive biases, our approach to training them typically consists of two phases: (i) fit the training data; and (ii) estimate the *generalization error* (the true error on the underlying population) by evaluating the model on holdout data. The difference between our fit on the training data and our fit on the test data is called the *generaliz.alion gap* and when this is large, we say that our models *overfil* to the training data. In extreme cases of overfitting, we might exactly fit the training data, even when the test error remains significant. And in the classical view, the interpretation is that our models are too complex, requiring that we either shrink the number of features, the number of nonzero parameters learned, or the size of the parameters as quantified. Recall the plot of model complexity compared with loss (Fig. 3.6.1) from Section 3.6.

However deep learning complicates this picture in counterintuitive ways. First, for classification problems, our models are typically expressive enough to perfectly fit every training example, even in datasets consisting of millions (Zhang *et al.*, 2021). ln the classical picture, we might think that this setting lies on the far right extreme of the model complexity axis, and that any improvements in generalization error must come by way of regularization, either by reducing the complexity of the model class, or by applying a penalty, severely constraining the set of values that our parameters might take. But that is where things start to get weird.

Strangely, for many deep learning tasks (e.g., image recognition and text classification) we are typically choosing among model architectures, all of which can achieve arbitrarily low training loss (and zero training error). Because all models under consideration achieve zero training error, *the only avenue.for.further gains is to reduce overfitting.* Even stranger, it is often the case that despite fitting the training data perfectly, we can actually *reduce the generalization error* further by making the model *even more expressive,* e.g., adding layers, nodes, or training for a larger number of epochs. Stranger yet, the pattern relating the generalization gap to the *complexity* of the model (as captured, for example, in the depth or width of the networks) can be non-monotonic, with greater complexity hurting at first but subsequently helping in a so-called "double-descent" pattern (Nakkiran *et al.,* 2021). Thus the deep learning practitioner possesses a bag of tricks, some of which seemingly restrict the model in some fashion and others that seemingly make it even more expressive, and all of which, in some sense, are applied to mitigate overfitting.

Complicating things even further, while the guarantees provided by classical learning theory can be conservative even for classical models, they appear powerless to explain why it is that deep neural networks generalize in the first place. Because deep neural networks are capable of fitting arbitrary labels even for large datasets, and despite the use of familiar methods such as £2 regularization, traditional complexity-based generalization bounds, e.g., those based on the VC dimension or Rademacher complexity of a hypothesis class cannot explain why neural networks generalize.

5..2 Inspiration from Nonparametrics

Approaching deep learning for the first time, it is tempting to think of them as parametric models. After all, the models *do* have millions of parameters. When we update the models, we update their parameters. When we save the models, we write their parameters to disk. However, mathematics and computer science are riddled with counterintuitive changes of perspective, and surprising isomorphisms between seemingly different problems. While neural networks clearly *have* parameters, in some ways it can be more fruitful to think of them as behaving like nonparametric models. So what precisely makes a model nonparametric? While the name covers a diverse set of approaches, one common theme is that nonparametric methods tend to have a level of complexity that grows as the amount of available data grows.

Perhaps the simplest example of a nonparametric model is the k-nearest neighbor algorithm (we will cover more nonparametric models later, for example in Section 11.2). Here, at training time, the learner simply memorizes the dataset. Then, at prediction time, when confronted with a new point x, the learner looks up the k nearest neighbors (the k points x; that minimize some distance d(x, x;)). When k = I, this algorithm is called I-nearest neighbors, and the algorithm will always achieve a training error of zero. That however, does not mean that the algorithm will not generalize. In fact, it turns out that under some mild conditions, the I-nearest neighbor algorithm is consistent (eventually converging to the optimal predictor).

Note that I-nearest neighbor requires that we specify some distance function *d,* or equivalently, that we specify some vector-valued basis function rp(x) for featurizing our data. For

any choice of the distance metric, we will achieve zero training error and eventually reach an optimal predictor, but different distance metrics d encode different inductive biases and with a finite amount of available data will yield different predictors. Different choices of the distance metric d represent different assumptions about the underlying patterns and the performance of the different predictors will depend on how compatible the assumptions are with the observed data.

In a sense, because neural networks are over-parametrized, possessing many more parameters than are needed to fit the training data, they tend to *interpolate* the training data (fitting it perfectly) and thus behave, in some ways, more like nonparametric models. More recent theoretical research has established deep connection between large neural networks and nonparametric methods, notably kernel methods. In particular, Jacot *et al.* (2018) demonstrated that in the limit, as multilayer perceptrons with randomly initialized weights grow infinitely wide, they become equivalent to (nonparametric) kernel methods for a specific choice of the kernel function (essentially, a distance function), which they call the neural tangent kernel. While current neural tangent kernel models may not fully explain the behavior of modern deep networks, their success as an analytical tool underscores the usefulness of nonparametric modeling for understanding the behavior of over-parametrized deep networks.

5..3 Early Stopping

While deep neural networks are capable of titting arbitrary labels, even when labels are assigned incorrectly or randomly (Zhang *et al.*, 2021), this capability only emerges over many iterations of training. A new line of work (Rolnick *et al.*, 2017) has revealed that in the setting of label noise, neural networks tend to fit cleanly labeled data first and only subsequently to interpolate the mislabeled data. Moreover, it has been established that this phenomenon translates directly into a guarantee on generalization: whenever a model has fitted the cleanly labeled data but not randomly labeled examples included in the training set, it has in fact generalized (Garg *et al.*, 2021).

Together these findings help to motivate *early i>tupping*, a classic technique for regularizing deep neural networks. Here, rather than directly constraining the values of the weights, one constrains the number of epochs of training. The most common way to determine the stopping criterion is to monitor validation error throughout training (typically by checking once after each epoch) and to cut off training when the validation error has not decreased by more than some small amount ε for some number of epochs. This is sometimes called a *patience criterion*. As well as the potential to lead to better generalization in the setting of noisy labels, another benefit of early stopping is the time saved. Once the patience criterion is met, one can terminate training. For large models that might require days of training simultaneously across eight or more GPUs, well-tuned early stopping can save researchers days of time and can save their employers many thousands of dollars.

Notably, when there is no label noise and datasets are *realizable* (the classes are truly separable, e.g., distinguishing cats from dogs), early stopping tends not to lead to significant improvements in generalization. On the other hand, when there is label noise, or intrinsic

variability in the label (e.g., predicting mortality among patients), early stopping is crucial. Training models until they interpolate noisy data is typically a bad idea.

5..4 Classical Regularization Methods for Deep Networks

In Chapter 3, we described several classical regularization techniques for constraining the complexity of our models. In particular, Section 3.7 introduced a method called weight decay, which consists of adding a regularization term to the loss function in order to penalize large values of the weights. Depending on which weight norm is penalized this technique is known either as ridge regularization (for $\pounds_2$ penalty) or lasso regularization (for an $\pounds_1$ penalty). In the classical analysis of these regularizers, they are considered as sufficiently restrictive on the values that the weights can take to prevent the model from fitting arbitrary labels.

In deep learning implementations, weight decay remains a popular tool. However, researchers have noted that typical strengths of $\pounds_2$ regularization are insufficient to prevent the networks from interpolating the data (Zhang *el al.*, 2021) and thus the benefits if interpreted as regularization might only make sense in combination with the early stopping criterion. Absent early stopping, it is possible that just like the number of layers or number of nodes (in deep learning) or the distance metric (in 1-nearest neighbor), these methods may lead to better generalization not because they meaningfully constrain the power of the neural network but rather because they somehow encode inductive biases that are better compatible with the patterns found in datasets of interests. Thus, classical regularizers remain popular in deep learning implementations, even if the theoretical rationale for their efficacy may be radically different.

Notably, deep learning researchers have also built on techniques first popularized in classical regularization contexts, such as adding noise to model inputs. In the next section we will introduce the famous dropout technique (invented by Srivastava *et al.* (2014)), which has become a mainstay of deep learning, even as the theoretical basis for its efficacy remains similarly mysterious.

5..5 Summary

Unlike classical linear models, which tend to have fewer parameters than examples, deep networks tend to be over-parametrized, and for most tasks are capable of perfectly fitting the training set. This *interpola1irm regime* challenges many hard fast-held intuitions. Functionally, neural networks look like parametric models. But thinking of them as nonparametric models can sometimes be a more reliable source of intuition. Because it is often the case that all deep networks under consideration are capable of fitting all of the training labels, nearly all gains must come by mitigating overfitting (closing the *generalization gap).* Paradoxically, the interventions that reduce the generalization gap sometimes appear to increase model complexity and at other times appear to decrease complexity. However, these methods seldom decrease complexity sufficiently for classical theory to explain the generalization of deep networks, and *why certain choices lead to improved generalization* remains for the most part a massive open question despite the concerted efforts of many brilliant researchers.

5..6 Exercises

1. In what sense do traditional complexity-based measures fail to account for generalization of deep neural networks?

2. Why might *early stopping* be considered a regularization technique?

3. How do researchers typically determine the stopping criterion?

4. What important factor seems to differentiate cases when early stopping leads to big improvements in generalization?

5. Beyond generalization, describe another benefit of early stopping.

106 Discussions [106].

• Dropout

Let's think briefly about what we expect from a good predictive model. We want it to perform well on unseen data. Classical generalization theory suggests that to close the gap between train and test performance, we should aim for a simple model. Simplicity can come in the form of a small number of dimensions. We explored this when discussing the monomial basis functions of linear models in Section 3.6. Additionally, as we saw when discussing weight decay (f_2 regularization) in Section 3.7, the (inverse) norm of the parameters also represents a useful measure of simplicity. Another useful notion of simplicity is smoothness, i.e., that the function should not be sensitive to small changes to its inputs. For instance, when we classify images, we would expect that adding some random noise to the pixels should be mostly harmless.

Bishop (1995) formalized this idea when he proved that training with input noise is equivalent to Tikhonov regularization. This work drew a clear mathematical connection between the requirement that a function be smooth (and thus simple), and the requirement that it be resilient to perturbations in the input.

Then, Srivastava *et al.* (2014) developed a clever idea for how to apply Bishop's idea to the internal layers of a network, too. Their idea, called *dropout*, involves injecting noise while computing each internal layer during forward propagation, and it has become a standard technique for training neural networks. The method is called *dropout* because we literally *drop out* some neurons during training. Throughout training, on each iteration, standard dropout consists of zeroing out some fraction of the nodes in each layer before calculating the subsequent layer.

To be clear, we are imposing our own narrative with the link to Bishop. The original paper on dropout offers intuition through a surprising analogy to sexual reproduction. The authors argue that neural network overfitting is characterized by a state in which each layer

relies on a specific pattern of activations in the previous layer, calling this condition *co-adaptation.* Dropout, they claim, breaks up co-adaptation just as sexual reproduction is argued to break up co-adapted genes. While such an justification of this theory is certainly up for debate, the dropout technique itself has proved enduring, and various forms of dropout are implemented in most deep learning libraries.

The key challenge is how to inject this noise. One idea is to inject it in an *unhiased* manner so that the expected value of each layer-while fixing the others--equals the value it would have taken absent noise. In Bishop's work, he added Gaussian noise to the inputs to a linear model. At each training iteration, he added noise sampled from a distribution with mean zero $E \sim N(0, cr^2)$ to the input x, yielding a perturbed point x' $=$ x $+ E$. Tn expectation, $E[x'] =$ x.

In standard dropout regularization, one zeros out some fraction of the nodes in each layer and then *dehiases* each layer by normalizing by the fraction of nodes that were retained (not dropped out). In other words, with *dropout probability p,* each intermediate activation his replaced by a random variable h' as follows:

$$h' = \begin{cases} 0 & \text{with probability } p \\ T'' = p & \text{otherwise} \end{cases} \tag{5.6.1}$$

By design, the expectation remains unchanged, i.e., $E[h'] = h$.

```
import torch
from torch import nn
from d21 import torch as d21
```

5..1 Dropout in Practice

Recall the **MLP** with a hidden layer and five hidden units from Fig. 5.1.1. When we apply dropout to a hidden layer, zeroing out each hidden unit with probability *p,* the result can be viewed as a network containing only a subset of the original neurons. **Tn** Fig. 5.6.1, *h2* and h_5 are removed. Consequently, the calculation of the outputs no longer depends on *h2* or *hs* and their respective gradient also vanishes when performing backpropagation. Tn this way, the calculation of the output layer cannot be overly dependent on any one element of *h1, ... ,hs.*

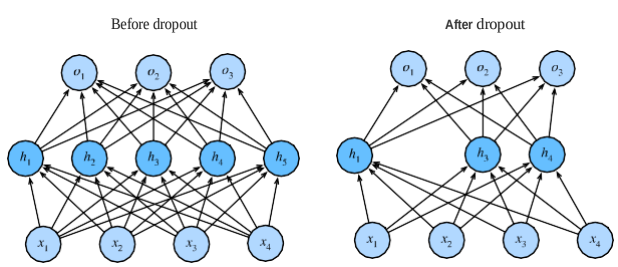


Fig. 5.6.1 MLP before and after dropout.

Typically, we disable dropout at test time. Given a trained model and a new example, we do not drop out any nodes and thus do not need to normalize. However, there are some exceptions: some researchers use dropout at test time as a heuristic for estimating the *uncertainty* of neural network predictions: if the predictions agree across many different dropout outputs, then we might say that the network is more confident.

5..2 Implementation from Scratch

To implement the dropout function for a single layer, we must draw as many samples from a Bernoulli (binary) random variable as our layer has dimensions, where the random variable takes value 1 (keep) with probability 1 - *p* and O (drop) with probability *p*. One easy way to implement this is to first draw samples from the uniform distribution U[O, 1]. Then we can keep those nodes for which the corresponding sample is greater than *p*, dropping the rest.

In the following code, we implement a dropout_layer function that drops out the elements in the tensor input X with probability dropout, rescaling the remainder as described above: dividing the survivors by 1. 0-dropout.

```
def dropout_layer(X, dropout):
    assert 0 <=dropout<= 1
    if dropout== 1: return torch.zeros_like(X)
    mask= (torch.rand(X.shape) > dropout).float()
    return mask* X / (1.0 - dropout)
```

We can test out the dropout_layer function on a few examples. In the following lines of code, we pass our input X through the dropout operation, with probabilities 0, 0.5, and 1, respectively.

```
X = torch.arange(16, dtype = torch.float32).reshape((2, 8))
print('dropout_p = 0:', dropout_layer(X, 0) )
print('dropout_p = 0.5:', dropout_layer(X, 0.5))
print('dropout_p = 1:', dropout_layer(X, 1))
```

```
dropout_p = 0: tensor([[ 0., 1., 2., 3., 4., 5., 6., 7.J,
       [ 8., 9., 10., 11., 12., 13., 14., 15.JJ)
dropout_p = 0.5: tensor([[ 0., 2., 0., 6., 8., 0., 0., 0.J,
       [16., 18., 20., 22., 24., 26., 28., 30.]])
dropout_p  = 1: tensor([[0., 0., 0., 0., 0., 0., 0., 0.J,
       [0., 0., 0., 0., 0., 0., 0., 0.]])
```

Defining the Model

The model below applies dropout to the output of each hidden layer (following the activation function). We can set dropout probabilities for each layer separately. A common choice is to set a lower dropout probability closer to the input layer. We ensure that dropout is only active during training.

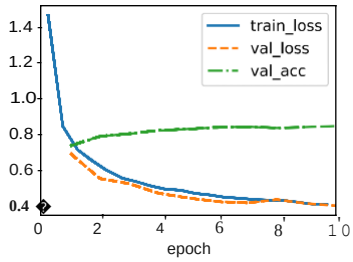
```
class DropoutMLPScratch(d21.Classifier):
    def __init__(self, num_outputs, num_hiddens_1, num_hiddens_2,
                 dropout_1, dropout_2, lr):
        super().__init__()
        self.save_hyperparameters()
        self.lin1  nn.Lazylinear(num_hiddens_1)
        self.lin2  nn.Lazylinear(num_hiddens_2)
        self.lin3  nn.Lazylinear(num_outputs)
        self.relu  nn.ReLU()

    def forward(self, X):
        H1 = self.relu(self.lin1(X.reshape((X.shape[0],-1))))
        if self.training:
            H1 = dropout_layer(H1, self.dropout_1)
        H2 = self.relu(self.lin2(H1))
        if self.training:
            H2 = dropout_layer(H2, self.dropout_2)
        return self.lin3(H2)
```

Training

The following is similar to the training of **MLPs** described previously.

```
hparams = {'num_outputs':10, 'num_hiddens_1':256, 'num_hiddens_2':256,
           'dropout_1':0.5, 'dropout_2':0.5, 'lr':0.1}
model= DropoutMLPScratch(**hparams)
data= d21.FashionMNIST(batch_size=256)
trainer= d21.Trainer(max_epochs= 10)
trainer.fit(model, data)
```



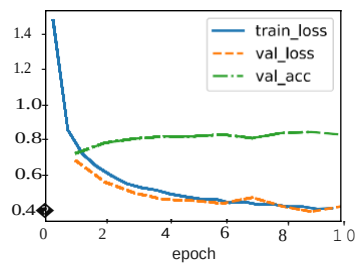
5..3 Concise Implementation

With high-level APls, all we need to do is add a Dropout layer after each fully connected layer, passing in the dropout probability as the only argument to its constructor. During training, the Dropout layer will randomly drop out outputs of the previous layer (or equivalently, the inputs to the subsequent layer) according to the specified dropout probability. When not in training mode, the Dropout layer simply passes the data through during testing.

```
class DropoutMLP(d21.Classifier):
    def __init__(self, num_outputs, num_hiddens_1, num_hiddens_2,
                 dropout_1, dropout_2, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net= nn.Sequential(
            nn.Flatten(), nn.Lazylinear(num_hiddens_1), nn.ReLU(),
            nn.Dropout(dropout_1), nn.Lazylinear(num_hiddens_2), nn.ReLU(),
            nn.Dropout(dropout_2), nn.Lazylinear(num_outputs))
```

Next, we train the model.

```
model= DropoutMLP(**hparams)
trainer.fit(model, data)
```



5..4 Summary

Beyond controlling the number of dimensions and the size of the weight vector, dropout is yet another tool for avoiding overfitting. Often tools are used jointly. Note that dropout is used only during training: it replaces an activation h with a random variable with expected value h.

5.6.S Exercises

1. What happens if you change the dropout probabilities for the first and second layers? In particular, what happens if you switch the ones for both layers? Design an experiment to answer these questions, describe your results quantitatively, and summarize the qualitative takeaways.

2. Increase the number of epochs and compare the results obtained when using dropout with those when not using it.

3. What is the variance of the activations in each hidden layer when dropout is and is not applied? Draw a plot to show how this quantity evolves over time for both models.

4. Why is dropout not typically used at test time?

5. Using the model in this section as an example, compare the effects of using dropout and

weight decay. What happens when dropout and weight decay are used at the same time? Are the results additive? Are there diminished returns (or worse)? Do they cancel each other out?

6. What happens if we apply dropout to the individual weights of the weight matrix rather than the activations?

7. Invent another technique for injecting random noise at each layer that is different from the standard dropout technique. Can you develop a method that outperforms dropout on the Fashion-MNIST dataset (for a fixed architecture)?

Discussions [107].

107 (II

S.7 Predicting House Prices on Kaggle

Now that we have introduced some basic tools for building and training deep networks and regularizing them with techniques including weight decay and dropout, we are ready to put all this knowledge into practice by participating in a Kaggle competition. The house price prediction competition is a great place to start. The data is fairly generic and do not exhibit exotic structure that might require specialized models (as audio or video might). This dataset, collected by De Cock (2011), covers house prices in Ames, Iowa from the period 2006-2010. Tt is considerably larger than the famous Boston housing dataset [108] of Harrison and Rubinfeld (1978), boasting both more examples and more features.

108

In this section, we will walk you through details of data preprocessing, model design, and hyperparameter selection. We hope that through a hands-on approach, you will gain some intuitions that will guide you in your career as a data scientist.

```
%matplotlib inline
import pandas as pd
import torch
from torch import nn
from d21 import torch as d21
```

5. 7.1 Downloading Data

Throughout the book, we will train and test models on various downloaded datasets. Here, we implement two utility functions for downloading and extracting zip or tar files. Again, we skip implementation details of such utility functions.

```
def download(url, folder, sha1_hash=None):
    """Download a file to folder and return the local filepath."""

def extract(filename, folder):
    """Extract a zip/tar file into folder."""
```